

Programmare con l'AI Agent

Come l'intelligenza artificiale sta cambiando
lo sviluppo software

Dalla scrittura manuale del codice

All'assistente intelligente

Cos'è un AI Agent per programmare

Non è un code-completion

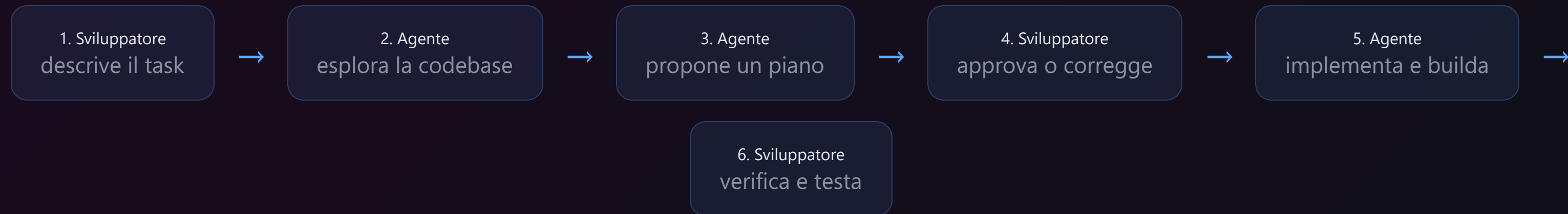
- Non suggerisce solo la prossima riga
- Lavora a **livello di progetto**, non di singolo file
- Legge decine di file in parallelo per capire l'architettura

È un assistente completo

- **Esplora** la codebase automaticamente
- **Pianifica** le modifiche necessarie
- **Implementa** scrivendo codice
- **Esegue build** e fixa gli errori
- **Fai il deploy** su cloud

Strumenti: **OpenCode** · Claude Code · GitHub Copilot X · Cursor · Windsurf

Come funziona il flusso di lavoro



Esempio concreto

Richiesta: "Aggiungi email per ritiro merchandising con QR code"

- Agente ha letto **16 file** in parallelo per capire modelli, endpoint e servizi esistenti
- Prodotto un piano con **10 file** da creare/modificare (modelli, endpoint, email, frontend)
- Implementato tutto in **20 minuti** — nuovo modello DB, API, email HTML con QR inline, pagina scanner QR

Impatto sulla produttività

113

file backend .cs

80+

endpoint API

6

iterazioni di sviluppo

Progetto **NoirFilmHub**: sviluppato da uno studente + AI agent in 3-4 mesi part-time

Sviluppo tradizionale vs AI Agent

- **Esplorare codebase**: ore/giorni → minuti
- **Trovare root cause di un bug**: ore di Stack Overflow → 3 messaggi
- **Deploy su Azure**: giorni di documentazione → agente esegue i comandi e fixa gli errori in tempo reale
- **Refactoring cross-file**: grep manuale rischioso → agente trova e modifica tutte le occorrenze

Limiti e rischi

Cosa può andare storto

- **Allucinazioni** : inventa metodi o API che non esistono
- **Overconfidence** : propone soluzioni complesse quando ne basta una semplice
- **Sensibilità al prompt** : richieste ambigue → risultati scadenti
- **Costo computazionale** : migliaia di token per sessione

Cosa NON sa fare

- Prendere **decisioni architetturali** autonome
- Capire il **dominio di business** (es. regole di rimborso)
- **Testare visivamente** l'interfaccia utente
- Dire **"questa feature non va fatta"**

Il codice generato **va sempre rivisto** —
come un **junior developer** molto veloce

Competenze per il futuro

Cosa resta fondamentale

- **Saper programmare** : per capire, correggere, validare
- **Architettura software** : l'agente propone, tu decidi
- **Pensiero critico** : valutare se il codice generato è corretto e sicuro

Nuove skill da sviluppare

- **Prompt engineering**: descrivere task in modo preciso e strutturato
- **AI supervision**: saper valutare, testare e iterare con l'agente
- **Lavoro aumentato**: usare l'AI come moltiplicatore, non come sostituto

"L'AI Agent è un **acceleratore, non un sostituto.**

Imparate a usarlo, non a dipenderne."